# Lecture 11: Introduction to Agentic AI

Language models, compound systems, tools, memory, and coordinated action

FYS4411 — Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

2026

# Motivation

A large language model maps a token sequence to a distribution over continuations. This operation is powerful, but it is not yet a persistent interaction process: the model does not independently maintain a task state, execute a tool, observe the consequence, or decide when the external objective has been satisfied.

Agentic AI places one or more models inside a computational system with explicit **state**, **actions**, **feedback**, **memory**, and **termination**. The model may choose among admissible actions, while conventional software validates and executes them.

The relevant object of study is therefore the complete system and its trajectory through an environment, not the linguistic quality of one model response.

Motivated by Cheng et al., *Exploring LLM-based Intelligent Agents*; BAIR, *Compound AI Systems*.

# Learning goals

After this lecture, you should be able to

- explain autoregressive language modelling, transformer attention, and post-training at a systems-relevant level;
- distinguish a foundation model, a compound workflow, an LLM-based agent, and a multi-agent system;
- formulate an agent loop using observations, internal state, objectives, actions, feedback, and stopping conditions;
- reason about retrieval, tool use, memory, planning, verification, and human approval as architectural components;
- evaluate whether agentic control is justified for a scientific or computational task.

# Organization

| Part | Central question |
|---|---|
| I. Language models | What computation produces an LLM output? |
| II. Compound AI systems | What components turn a model into a useful system? |
| III. Agentic flow | How are actions selected, executed, evaluated, and revised? |
| IV. Multi-agent systems | When does specialization justify coordination? |
| V. Applications | Where do present systems obtain measurable feedback? |
| VI. Evaluation and safety | What evidence is required before deployment or scientific use? |

Motivated by Agents for Science curriculum, Lectures 1–13.

# Part I: How large language models work

A concise computational foundation

# Autoregressive language modelling

For a token sequence $x_{1:T}$, an autoregressive model factorizes the joint probability as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Training minimizes empirical negative log-likelihood,

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} \log p_\theta\left(x_t^{(n)} \mid x_{<t}^{(n)}\right).$$

At inference time, the same conditional distribution is queried repeatedly. One token is selected, appended to the context, and used to condition the next step.

Motivated by Brown et al., *Language Models are Few-Shot Learners*.

# Tokenization defines the model's discrete interface

Text is first transformed into a sequence of token identifiers

$$\texttt{"agentic AI"} \longrightarrow (i_1, i_2, \ldots, i_m),$$

where tokens may be words, subwords, punctuation, bytes, or mixtures of these units.

**Consequences for computation**

- Cost and context length are measured in tokens.
- Rare strings may require several tokens.
- Tool schemas and code consume the same context budget as prose.

**Consequences for meaning**

- The model does not receive characters or words directly.
- Identical concepts may have different token decompositions.
- Token boundaries are engineering choices, not semantic primitives.

# Embeddings and positional information

Each identifier $i_t$ indexes a learned vector $e_t \in \mathbb{R}^d$. A positional representation $p_t$ is added or incorporated so that sequence order is available:

$$h_t^{(0)} = e_t + p_t.$$

The transformer then constructs contextual representations $h_t^{(\ell)}$ through repeated attention and feed-forward layers.


Motivated by Vaswani et al., *Attention Is All You Need*.

# The transformer block

For each layer $\ell$, a decoder-only transformer applies masked multi-head self-attention followed by a position-wise feed-forward network, with residual connections and normalization:

$$\tilde{H}^{(\ell)} = \mathrm{Norm}\big(H^{(\ell-1)} + \mathrm{MHA}(H^{(\ell-1)})\big)\,,$$

$$H^{(\ell)} = \mathrm{Norm}\big(\tilde{H}^{(\ell)} + \mathrm{FFN}(\tilde{H}^{(\ell)})\big)\,.$$


Motivated by Vaswani et al. (2017).

# Scaled dot-product attention

For input matrix $H \in \mathbb{R}^{T \times d}$, learned projections define

$$Q = HW_Q, \qquad K = HW_K, \qquad V = HW_V.$$

One attention head computes

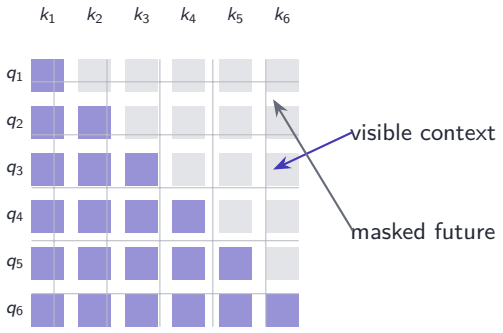
$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

where $M_{ij} = -\infty$ when position $i$ is not permitted to attend to position $j$.
The weights in each row of the softmax define a data-dependent mixture of value vectors. Multi-head attention repeats this operation with different learned projections and concatenates the results.

Motivated by Vaswani et al. (2017).

# Causal masking supports next-token prediction

During training, all target positions can be processed in parallel, but token $x_t$ must not use information from $x_{t+1:T}$.



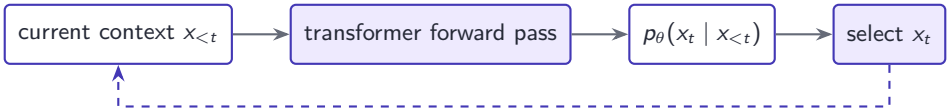

The mask makes the network a conditional sequence model rather than a bidirectional encoder.

# Generation is a closed recurrence over context

Given prompt $x_{1:t-1}$, generation repeats

$$z_t = f_\theta(x_{<t}), \qquad p_t = \mathrm{softmax}(z_t), \qquad x_t \sim D(p_t),$$

where $D$ denotes a decoding rule.




The loop terminates at a stop token, a length limit, or an external control condition. The model does not independently know whether a real-world task has been completed.

# Decoding changes the distribution of continuations

Temperature rescales logits,

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)},$$

while top-$k$ or nucleus sampling removes low-probability candidates before sampling.

| **Rule** | **Main effect** | **Systems implication** |
|---|---|---|
| Greedy / beam | concentrates on high likelihood | repeatable, but not necessarily correct |
| Temperature | changes entropy | controls diversity of trajectories |
| Top-$p$ / top-$k$ | truncates the tail | avoids very unlikely actions |
| Multiple samples | explores alternatives | raises cost; requires selection or verification |

# Pretraining yields broad but implicit capabilities

Large-scale next-token training produces representations that support language, code, question answering, and task adaptation from context. Empirical scaling laws show predictable improvement of cross-entropy loss with model size, data, and training compute over broad regimes.

However, pretraining optimizes predictive likelihood, not truth, task completion, calibrated uncertainty, or compliance with a user's operational constraints.

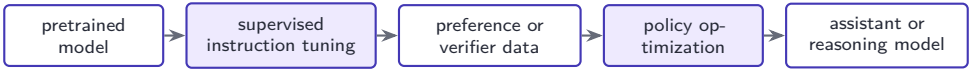
## Systems consequence

The model can be a high-capacity component in a controller, but its pretraining objective is not itself an objective for reliable action in an external environment.

Motivated by Kaplan et al., *Scaling Laws*; Brown et al. (2020).

# Post-training shapes the model's policy

Instruction-tuned systems are commonly obtained through a sequence such as




For RLHF, a learned reward model $r_\phi(x, y)$ approximates human preferences and the policy is optimized under a regularization constraint, schematically

$$\max_\theta \ \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r_\phi(x, y)] - \beta\, D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}}).$$

Post-training changes which capabilities are elicited and how outputs are distributed; it does not make tool outputs or factual claims intrinsically valid.

Motivated by Ouyang et al., *Training language models to follow instructions with human feedback*.

# In-context learning is inference, not parameter updating

Instructions, demonstrations, retrieved passages, and tool results modify the conditional distribution by changing the input sequence:

$$p_\theta(y \mid x) \longrightarrow p_\theta(y \mid \text{instruction}, \text{examples}, x).$$

**What changes**

- the active evidence and task framing;
- the apparent input-output mapping;
- the probability of particular formats and actions.

**What does not change**

- model parameters $\theta$;
- the training corpus;
- persistent memory after the context is discarded.

Motivated by Brown et al. (2020).

# Context is finite working state

The context window must simultaneously hold instructions, user data, retrieved evidence, tool definitions, tool results, intermediate state, and generated reasoning or plans.
If the context has maximum length $C$, the controller must satisfy

$$L_{\text{instructions}} + L_{\text{state}} + L_{\text{evidence}} + L_{\text{tools}} + L_{\text{output}} \leq C.$$

Long contexts create three distinct problems:

- **capacity**: relevant information may not fit;
- **selection**: irrelevant information competes for attention;
- **integrity**: untrusted content may be mistaken for instructions.

Persistent state therefore requires storage, retrieval, and update mechanisms outside a single model invocation.

# An LLM is not yet an agent

| | **Language model** | **Agentic system** |
|---|---|---|
| Input | finite token context | observations, state, goal, constraints |
| Output | token distribution / message | typed action or final result |
| Memory | parameters and active context | explicit working and persistent state |
| Environment | absent from model definition | tools, software, humans, laboratories |
| Feedback | next-token loss during training | execution results and task-level tests |
| Termination | sequence-level stop condition | verified completion or safe abort |

An LLM becomes agentic only through its placement inside a system that interprets selected outputs as actions with external consequences.

Motivated by Cheng et al. (2024); Agents for Science curriculum.

# Capabilities and limitations must be separated

**Capabilities useful to agents**

- interpret heterogeneous natural-language observations;
- generate code and structured tool arguments;
- adapt to new tasks from instructions and examples;
- propose decompositions and alternative hypotheses.

**Limitations relevant to control**

- factual claims may be unsupported;
- outputs vary with context and sampling;
- internal confidence is not a calibrated task-success estimate;
- the model does not enforce permissions or side-effect constraints.

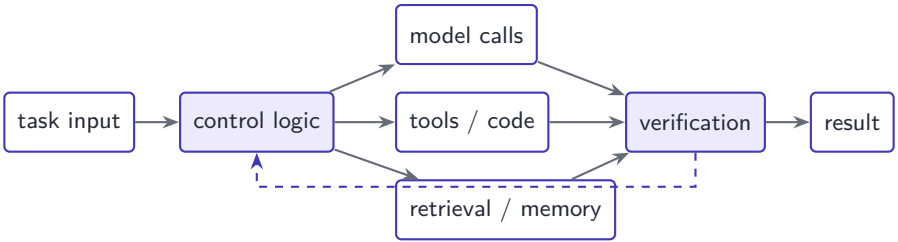
The purpose of the surrounding system is not to remove all model error. It is to make useful actions possible while rendering failures observable, bounded, and recoverable.

# Part II: Compound AI systems

Models embedded in retrieval, tools, memory, and control logic

# From a model to a compound AI system

A **compound AI system** addresses a task using multiple interacting components, which may include repeated model calls, retrievers, databases, external tools, verifiers, and conventional software.




The model is a component of the system; system-level performance depends on interfaces and orchestration as well as model quality.

Motivated by BAIR, *The Shift from Models to Compound AI Systems*.

# Why use compound systems?

1. **Dynamic knowledge.** Retrieval and APIs expose information that is newer, private, or too large for model parameters and prompt context.
2. **Grounded computation.** Calculators, code, simulators, databases, and instruments perform operations whose outputs can be inspected independently.
3. **Control and assurance.** Validation, filters, permissions, and approval gates can be enforced outside the generative model.
4. **Adaptive computation.** Difficult cases can receive more model calls, search branches, or verification effort than easy cases.
5. **Modularity.** Components can be evaluated and replaced without retraining the entire foundation model.

The benefit must be compared with additional latency, cost, integration complexity, and new failure modes.

Motivated by BAIR (2024); Kapoor et al., *AI Agents That Matter*.

# A model call, workflow, and agent differ in control allocation

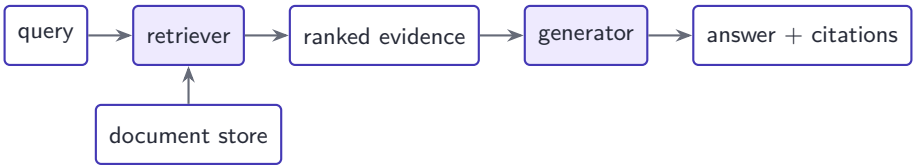
| System form | Control path | Typical example |
|---|---|---|
| Single model call | one fixed invocation | summarize a supplied document |
| Deterministic workflow | software fixes all transitions | extract, validate, transform, and store |
| Model-routed workflow | model chooses among fixed branches | classify request and select one specialist |
| Agent | model repeatedly selects next actions from state and feedback | inspect repository, edit, test, and revise |
| Multi-agent system | several decision makers interact through a protocol | planner, executor, critic, and coordinator |

Agentic control is therefore a property of the surrounding control graph, not a property of a text string.

# Retrieval-augmented generation

For a query $q$, a retriever selects documents $z_{1:k}$ and the generator conditions on both query and evidence:

$$p(y \mid q) \approx \sum_{z \in \mathrm{TopK}(q)} p_\eta(z \mid q)\, p_\theta(y \mid q, z).$$




Retrieval does not guarantee correctness. Failures may arise from query formulation, missing documents, ranking, context integration, or unsupported synthesis.

Motivated by Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.

# Tools convert model output into typed operations

A tool interface exposes a name, a semantic description, an argument schema, and a result schema. A model proposes an invocation such as

$$a_t = \texttt{search\_papers}(\{\texttt{query} : q,\ \texttt{year\_from} : 2024\}) .$$

The runtime then performs a separate sequence:


The model proposes; the runtime validates and executes. Schema conformance alone does not establish that an action is safe, authorized, or appropriate.

Motivated by Model Context Protocol documentation; Agents for Science, Lecture 5.

# Code execution creates an external testable substrate

Code tools allow the system to implement hypotheses, run simulations, query data, and inspect artifacts. A typical loop is

$$\text{specification} \rightarrow \text{code} \rightarrow \text{execution} \rightarrow \text{test result} \rightarrow \text{revision}.$$

**Why it is powerful**

- intermediate results are observable;
- tests can provide objective feedback;
- artifacts can be reproduced and reviewed;
- search can be automated over candidate programs.

**Why it is dangerous**

- generated code may be incorrect or destructive;
- execution may expose secrets or networks;
- tests may be incomplete or mis-specified;
- successful execution is not scientific validity.

Sandboxing, permission boundaries, timeouts, and artifact inspection are part of the system definition.

# External memory is an indexed state system

Memory is not simply a longer transcript. It requires a write policy, representation, index, retrieval policy, and deletion or update policy.

| Layer | Typical content | Principal risk |
|---|---|---|
| Working state | current plan, subgoal, recent observations | context overload or accidental omission |
| Episodic memory | prior actions, outcomes, trajectories | copying obsolete or failed strategies |
| Semantic memory | facts, summaries, structured knowledge | unsupported compression and staleness |
| Artifact store | files, code, datasets, checkpoints | identity, version, and provenance errors |

Retrieval from memory is itself an action that can fail. Persisted content should not automatically be trusted as an instruction.

Motivated by Cheng et al. (2024), agent memory survey.

# Verification turns outputs into feedback

A verifier maps a candidate artifact $y$ and task specification $s$ to diagnostic evidence

$$v(y, s) \to (\text{status}, \text{score}, \text{diagnostics}).$$

Useful verifiers include unit tests, type checkers, theorem provers, constraint solvers, simulators, laboratory measurements, and human review.

## Independence matters

A second language-model judgment is not automatically independent evidence. Verification is strongest when it uses a different information source, formal rule, executable test, or measurement process.

Feedback should localize failure where possible. A scalar score may rank candidates, whereas structured diagnostics can support targeted revision.

# System-level optimization has several budgets

Let $S$ be task success, $C$ monetary or compute cost, $L$ latency, and $R$ operational risk. A system designer seeks a policy and architecture $\mathcal{A}$ satisfying a multi-objective criterion such as

$$\max_{\mathcal{A}} \mathbb{E}[S] \quad \text{subject to} \quad \mathbb{E}[C] \leq C_{\max},\ \Pr(L > L_{\max}) \leq \epsilon,\ R \leq R_{\max}.$$

Increasing model calls, retrieval depth, search width, or critic iterations may improve success but also increases the opportunity for inconsistent state, security exposure, and unbounded runtime.
The question is not whether a component improves average benchmark accuracy in isolation. It is whether the complete system improves the required operating point.

Motivated by Kapoor et al. (2024); BAIR (2024).

# Part III: Agentic AI flow

Goal-directed action under feedback

# A working definition of an LLM-based agent

An LLM-based agent is a system in which a model repeatedly selects actions from an admissible action set using observations, internal state, an objective, and constraints.
One useful abstraction is

$$a_t \sim \pi_\theta(\cdot \mid o_{\le t}, m_t, g, c), \qquad o_{t+1} = \mathcal{E}(a_t), \qquad m_{t+1} = U(m_t, a_t, o_{t+1}),$$

where $\mathcal{E}$ is the external environment and $U$ is the system's state-update rule.

- The policy may be a single model call or a compound inference procedure.
- Actions may be informational, computational, communicative, or mutating.
- Autonomy is determined by which actions the system may execute without approval.

Motivated by Cheng et al. (2024); Russell and Norvig's agent-environment abstraction.

# Relation to a partially observable decision process

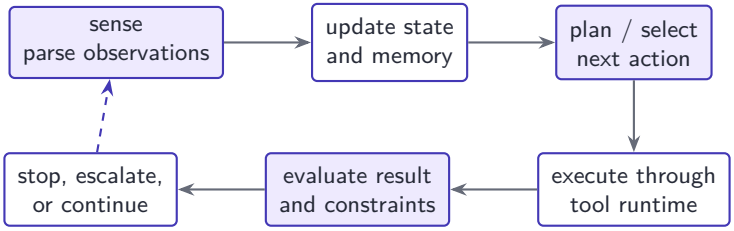
Agentic systems often operate under partial observability. A formal model is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma),$$

with latent state $s_t$, observation $o_t \sim Z(\cdot \mid s_t)$, action $a_t$, and transition $s_{t+1} \sim T(\cdot \mid s_t, a_t)$.
Unlike conventional RL, many LLM agents are not trained online to maximize a scalar reward in the deployment environment. Their policy is assembled from pretrained models, prompts, tools, memory, and software control. The POMDP view remains useful as a **diagnostic language**:

- What information is observable?
- What state must be inferred or stored?
- Which actions change the environment?
- Which feedback signal indicates progress?

# The sense–plan–act–evaluate loop




Each transition should have an explicit data contract. A failure to distinguish observation, state, instruction, and action is a frequent source of agent error.
The loop may be implemented sequentially, as a state machine, as a graph with conditional branches, or as search over several candidate trajectories.

Motivated by Agents for Science curriculum, Lecture 1; ReAct.

# Observations are not equivalent to facts

The observation $o_t$ is the information delivered to the controller at step $t$. It may contain user messages, retrieved passages, tool results, file contents, sensor readings, or messages from other agents.
An observation should be represented with metadata

$$o_t = (\text{content}, \text{source}, \text{time}, \text{authority}, \text{integrity}, \text{uncertainty}).$$

- **Partial**: relevant state may remain unobserved.
- **Noisy**: measurements or retrieval may be incorrect.
- **Stale**: the environment may have changed.
- **Adversarial**: external text may contain instructions intended to redirect the model.

The controller must treat observations as data to interpret, not automatically as trusted commands.

# Internal state is a designed sufficient statistic

Ideally, the maintained state $m_t$ contains the information required for the next decision without reproducing the entire interaction history:

$$m_t = \left(g_t,\ P_t,\ B_t,\ A_t^{\mathrm{done}},\ F_t,\ C_t\right),$$

where $g_t$ is the active subgoal, $P_t$ the plan, $B_t$ remaining budgets, $A_t^{\mathrm{done}}$ completed actions, $F_t$ artifacts and evidence, and $C_t$ constraints.

**Good state representation**

- explicit and typed;
- versioned and inspectable;
- updated after every action;
- separates claims from evidence.

**Weak state representation**

- unbounded conversation history;
- implicit completion status;
- overwritten artifacts;
- unresolved contradictions.

# Objectives must be operationalized

Natural-language goals are generally incomplete specifications. The system should translate a goal $g$ into

$$\mathcal{G} = (\text{target state}, \text{acceptance tests}, \text{constraints}, \text{budget}, \text{deadline}, \text{authority}).$$

For example, "review this analysis" does not specify whether the agent may edit files, run code, access external data, or publish changes.

## Design principle

Safety and authority constraints should be enforced by software and permissions. A prompt may communicate policy to the model, but it is not an access-control mechanism.

A goal is operational only when the system can determine success, failure, or the need for human judgment from available evidence.

# The action space determines practical autonomy

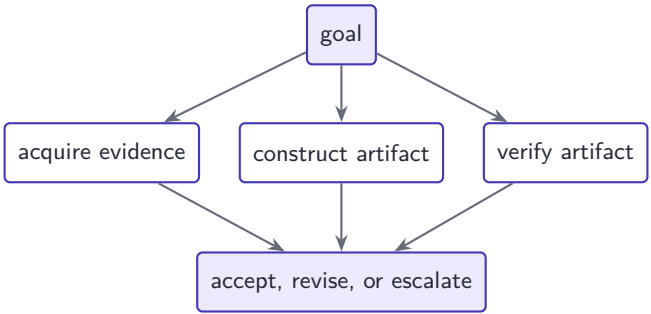
| Action class | Examples | Relevant controls |
|---|---|---|
| Informational | search, retrieve, read, inspect | data access, provenance, rate limits |
| Computational | execute code, simulate, optimize | sandbox, resource limits, reproducibility |
| Communicative | ask, report, delegate, message | recipient verification, disclosure policy |
| Mutating | write, submit, purchase, actuate | approval, least privilege, rollback |
| Terminal | finish, abort, escalate | acceptance tests, failure reporting |

If $\mathcal{A}_{\text{allowed}}(m_t)$ is determined by state and policy, the model should select only from this filtered set:

$$a_t \sim \pi_\theta(\cdot \mid m_t, o_t), \qquad a_t \in \mathcal{A}_{\text{allowed}}(m_t).$$

# Planning decomposes goals into conditional structure

A plan is not merely a list of steps. It should identify dependencies, information requirements, decision points, verification operations, and recovery paths.




Planning is useful when it reduces myopic action selection, exposes missing prerequisites, and creates checkpoints. It is harmful when the initial plan is treated as authoritative despite new observations.

# Replanning is triggered by state mismatch

Let a plan predict observation distribution $\hat{p}(o_{t+1} \mid m_t, a_t)$. Replanning is appropriate when the actual observation is incompatible with the plan, a constraint changes, or execution fails:

$$\text{surprise}(o_{t+1}) = -\log \hat{p}(o_{t+1} \mid m_t, a_t) > \delta.$$

In practice, triggers are often rule-based:

- tool status is failure or partial completion;
- a verifier rejects the artifact;
- newly retrieved evidence contradicts an assumption;
- cost, time, or iteration budget is exhausted;
- the next action requires authority not currently granted.

Replanning should update state and assumptions before asking the model to propose a new path.

# ReAct interleaves reasoning and acting

ReAct introduced a simple pattern in which model-generated reasoning and environment actions alternate:

| Phase | Example trace |
|---|---|
| Reason | The current evidence does not identify the paper's publication date. |
| Action | `search(query = title, domain = arxiv.org)` |
| Observation | Structured search result with title, authors, identifier, and date. |
| Reason | The source resolves the date; compare the abstract with the claim. |
| Action | `open(arxiv_id)` |

The important systems contribution is not unrestricted chain-of-thought text. It is the coupling of intermediate decision state with observations obtained through actions.
Actions ground the trajectory; reasoning organizes what to do next. Either component can still be wrong.

Motivated by Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*.

# Tool selection is a constrained value-of-information decision

For an informational action $a$, a rational controller compares expected decision improvement with execution cost and risk:

$$a_t^* = \arg \max_{a \in \mathcal{A}_{\text{allowed}}} \Big( \mathbb{E}[V(m_{t+1}) \mid m_t, a] - V(m_t) - \lambda C(a) - \rho R(a) \Big).$$

The model typically approximates this choice linguistically rather than evaluating the expression numerically. Relevant questions include

- Does the tool's declared capability match the current information need?
- Is the result likely to change the next decision?
- Are arguments valid and minimal?
- Is a less powerful or less expensive tool sufficient?
- Can the result be independently checked?

# The execution boundary must be explicit


**Model responsibility**

- select a declared tool;
- supply typed arguments;
- interpret status and returned data.

**Runtime responsibility**

- enforce schema and permissions;
- execute with bounded resources;
- preserve logs, outputs, and errors.

No model response should directly bypass this boundary for a consequential action.

# Tool results require normalization and provenance

The raw result $r_t$ should be converted to an observation with explicit status:

$$N(r_t) = (\texttt{success} \mid \texttt{partial} \mid \texttt{failure}, \text{data}, \text{diagnostics}, \text{provenance}).$$

Normalization should

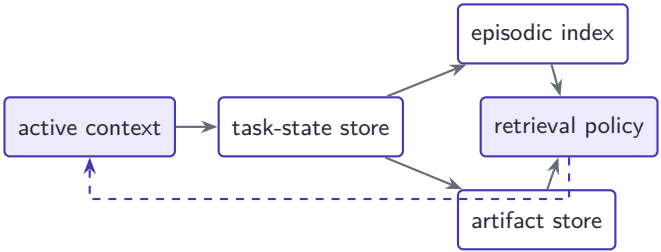
- separate tool data from text that might be interpreted as instructions;
- retain identifiers, versions, timestamps, and units;
- distinguish missing values from negative results;
- summarize large outputs while keeping a reference to the complete artifact;
- expose partial completion and retryability rather than returning only prose.

Lossy summaries can corrupt state. The system should preserve the underlying evidence required for later verification.

# Memory operations are part of the agent policy

At each step, the system may decide what to write, retrieve, update, or forget:

$$m_{t+1} = U\big(m_t, \text{write}(o_{t+1}), \text{retrieve}(q_t), \text{delete}(d_t)\big).$$




Writing everything produces noisy memory; writing only model summaries risks losing evidence. Memory design should be evaluated as a retrieval system, not as a metaphor for human cognition.

# Reflection is revision conditioned on feedback

A useful reflection loop has four distinct objects:

$$y^{(k)} = \text{propose}(m), \quad f^{(k)} = \text{test}(y^{(k)}), \quad d^{(k)} = \text{diagnose}(f^{(k)}), \quad y^{(k+1)} = \text{revise}(y^{(k)}, d^{(k)}).$$

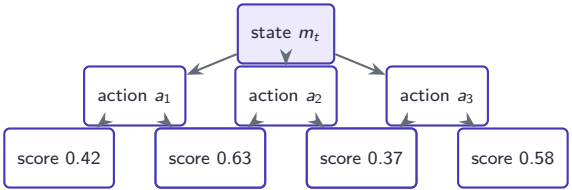
## Necessary condition

The feedback $f^{(k)}$ must contain information not already exhausted by the proposal process. Repeated self-critique without an external test may reproduce or amplify the original error.

Reflection can improve formatting, local consistency, code, and test-driven artifacts. It cannot establish an empirical claim without appropriate data or measurement.

# Search explores multiple candidate trajectories

Instead of committing to one next action, a system may sample candidates, evaluate them, and expand promising branches:




Search width, depth, and selection rule determine test-time compute. Tree search is useful only when evaluators rank partial trajectories with enough fidelity to guide expansion.
For open-ended scientific tasks, the evaluator is often the limiting component rather than the generator.

Motivated by AI Scientist-v2; Google AI co-scientist.

# Test-time compute is an adaptive resource

Let $K$ denote candidate generations and $J$ verifier calls. A compound inference policy may allocate $(K, J)$ as a function of task state:

$$(K_t, J_t) = \alpha(m_t, \widehat{u}_t, B_t),$$

where $\widehat{u}_t$ is estimated uncertainty and $B_t$ the remaining budget.
Common strategies include

- self-consistency over multiple samples;
- best-of-$N$ selection with a reward model or executable test;
- iterative refinement until a verifier threshold is reached;
- tree search over plans, experiments, or programs;
- escalation to a stronger model or human expert for selected states.

More calls are not automatically beneficial: correlated errors and weak evaluators can make additional computation confidently wrong.

# Three common control architectures

| Pattern | Control structure | Appropriate use |
|---|---|---|
| Router | classify state and choose one fixed branch | heterogeneous but well-defined request classes |
| Plan–execute | construct a plan, execute steps, replan on triggers | long tasks with dependencies and observable progress |
| Evaluator–optimizer | generate candidate, test, diagnose, revise | artifacts with informative automated feedback |
| Search | expand and rank several trajectories | high-value tasks with reliable partial evaluation |

The architecture should encode deterministic guarantees directly. Model-directed control should be reserved for choices that require semantic adaptation.

Motivated by Compound systems literature; ReAct; agent workflow frameworks.

# Termination is a decision, not a formatting event

The controller chooses among $\{\texttt{continue}, \texttt{finish}, \texttt{abort}, \texttt{escalate}\}$ using state and verification evidence. A termination predicate can be written

$$\text{done}(m_t) = \mathbb{K}\{V_{\text{required}}(F_t) = \texttt{pass}\} \vee \mathbb{K}\{B_t \leq 0\} \vee \mathbb{K}\{C_t \text{ violated}\}.$$

- **Finish**: acceptance tests pass and required artifacts exist.
- **Abort**: continued execution is unsafe or impossible.
- **Escalate**: progress requires authority or judgment outside the system.
- **Continue**: the next action has positive expected value within budget.

Fluent final prose is neither necessary nor sufficient for task completion.

# Human oversight belongs at decision boundaries

| Boundary | Human responsibility | System evidence required |
|---|---|---|
| Goal formation | define scope and acceptable evidence | task specification and exclusions |
| High-impact action | authorize irreversible consequences | proposed action, target, and rollback plan |
| Ambiguous evidence | apply domain judgment and values | conflicting sources and uncertainty |
| Final acceptance | own scientific or organizational conclusion | trajectory, artifacts, tests, provenance |

Human-in-the-loop design is not simply "ask for approval often." Approval requests should occur where human information, authority, or accountability materially changes the decision.

# Autonomy is assigned per action class


One system may retrieve documents autonomously, execute code only in a sandbox, require approval before sending messages, and prohibit purchases entirely.

Therefore, “the agent is autonomous” is underspecified. A formal description should state the action set, policy constraints, approval rules, budgets, and maximum uninterrupted horizon.

# When is an agent justified?

Agentic control is most defensible when

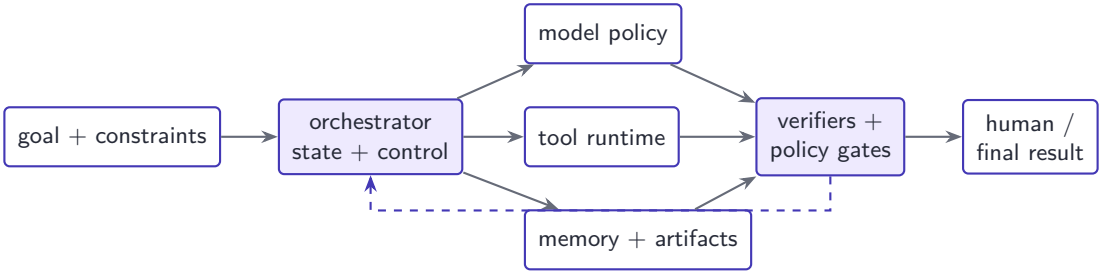
- the path to the objective cannot be fully specified before observing intermediate results;
- the environment exposes tools or measurements that provide task-relevant feedback;
- progress requires semantic interpretation across heterogeneous observations;
- actions are reversible, sandboxed, or protected by approval and policy gates;
- the expected value of adaptation exceeds additional cost, latency, and monitoring burden.

## Counterfactual test

If a fixed workflow using the same models and tools meets the requirement, the agentic design must demonstrate a measurable advantage to justify its larger control and failure surface.

Motivated by BAIR (2024); Kapoor et al. (2024).

# A reference architecture separates responsibilities




The orchestrator owns state transitions and budgets. The model proposes semantic decisions. The runtime owns execution. Verifiers and policy gates determine whether results can advance the state.

Motivated by Compound AI systems; MCP; LLM-agent architecture surveys.

# Errors can compound across the trajectory

If a step succeeds with conditional probability $p_t$, a naive independent approximation gives trajectory success

$$P(\text{all } T \text{ steps succeed}) \approx \prod_{t=1}^{T} p_t.$$

Even $p_t = 0.98$ yields $0.98^{30} \approx 0.545$ over thirty required steps. Real failures are not independent: one incorrect observation can corrupt state and all later decisions.


Checkpoints, independent tests, bounded horizons, and state inspection are methods for breaking this chain.

# Part IV: Multi-agent systems

Specialization, communication, and coordination

# Definition and system model

A multi-agent system contains a set of decision makers $\mathcal{I} = \{1, \dots, n\}$ that interact through messages and a shared or partially shared environment.

$$a_t^i \sim \pi_i(\cdot \mid o_{\le t}^i, m_t^i, g_i), \qquad \mu_t^{i \to j} \in \mathcal{M}_{ij}.$$

An LLM-based multi-agent system may use the same foundation model for every $\pi_i$ while varying prompts, tools, memory, observations, and authority.

- **Heterogeneity**: agents have distinct capabilities or information.
- **Communication**: agents exchange messages or write shared state.
- **Coordination**: a protocol determines allocation, synchronization, and termination.
- **Joint objective**: local actions are evaluated against system-level success.

Multiple model calls are not by themselves a multi-agent architecture; the system must define interacting roles and control.

Motivated by Cheng et al. (2024); AutoGen.

# Why introduce multiple agents?

The main architectural motivations are

1. **specialization**: different contexts, tools, or domain instructions can be isolated;
2. **parallelism**: separable searches or experiments can be executed concurrently;
3. **independent verification**: a critic can inspect an artifact without inheriting the complete generation trace;
4. **modularity**: roles expose interfaces that can be evaluated and replaced;
5. **organizational analogy**: complex workflows can be represented as allocation among planners, executors, reviewers, and coordinators.

These benefits appear only when the work is genuinely separable. Otherwise communication increases context, cost, and opportunities for inconsistency.

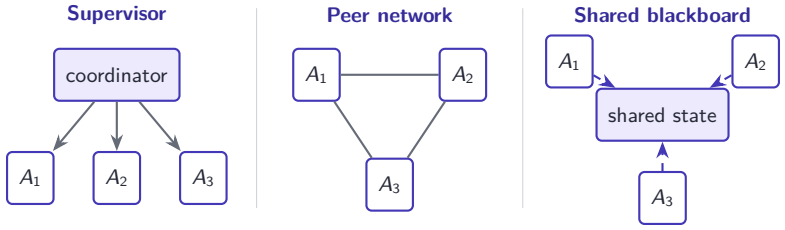
# Role specialization defines responsibility

| Role | Primary responsibility | Acceptance evidence |
|---|---|---|
| Planner | decompose and allocate work | dependency-respecting plan |
| Researcher | retrieve and synthesize evidence | cited, provenance-aware evidence set |
| Executor | implement code or operate tools | artifact plus execution record |
| Experimenter | specify and run protocols | measurements and metadata |
| Critic | localize error and test claims | diagnostics independent of generator |
| Coordinator | integrate state and terminate | consistent system state and final tests |

Roles should determine permissions and outputs, not merely conversational personas. Every role needs a bounded interface and an owner for unresolved failures.

Motivated by AutoGen; Google AI co-scientist; Virtual Lab.

# Coordination topologies




- Supervisors simplify allocation but create a central bottleneck.
- Peer networks support distributed negotiation but complicate state consistency.
- Shared blackboards separate communication from durable system state.

# Communication is a protocol, not free-form dialogue

A message should have a schema

$$\mu = (\text{sender}, \text{recipient}, \text{type}, \text{task id}, \text{content}, \text{artifacts}, \text{provenance}, \text{deadline}).$$

Useful message types include `request`, `proposal`, `result`, `critique`, `acknowledgment`, and `escalation`.

**Protocol responsibilities**

- route to the correct recipient;
- prevent duplicate ownership;
- define what must be acknowledged;
- bound context and communication frequency.

**Shared-state responsibilities**

- record authoritative task status;
- version artifacts and decisions;
- expose unresolved conflicts;
- support recovery after agent failure.

Motivated by AutoGen; federated scientific-agent architectures.

# Debate and critique require information diversity

Suppose agents produce candidate beliefs $b_i$ and a coordinator aggregates them. Multiple agents improve the decision only if their errors are not fully correlated and the aggregation rule uses evidence effectively.
For binary independent errors with accuracy $p > 1/2$, majority voting improves with $n$; correlated errors sharply reduce this gain. LLM agents frequently share the same model, prompt conventions, and retrieved evidence, so independence cannot be assumed.

## Conditions for useful critique

Distinct tools or evidence; blinded generation where appropriate; an explicit rubric; a decision rule; and a mechanism for preserving unresolved disagreement.

Debate can diversify hypothesis search. It cannot replace empirical or formal verification when the participants share the same unsupported premise.

Motivated by AI co-scientist; Virtual Lab; multi-agent evaluation literature.

# Multi-agent systems introduce characteristic failures

Cemri et al. identify fourteen failure modes grouped into three broad categories:

| Category | Examples | System consequence |
|---|---|---|
| Specification and design | incomplete roles, missing information, poor task allocation | agents cannot produce compatible outputs |
| Inter-agent misalignment | ignored messages, withholding, duplicated work, conflicting goals | local progress does not compose |
| Verification and termination | no acceptance owner, premature stopping, endless loops | system cannot establish completion |

Better prompts alone do not resolve ownership, protocol, and state-management failures. Multi-agent performance is a systems-engineering problem.

Motivated by Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (2025).

# When is a multi-agent design justified?

A useful decision inequality is

$$\Delta V_{\text{specialization}} + \Delta V_{\text{parallelism}} + \Delta V_{\text{verification}} > C_{\text{communication}} + C_{\text{coordination}} + R_{\text{misalignment}}.$$

Use multiple agents when roles require distinct permissions, tools, context windows, expertise, or independent tests; when work can proceed in parallel; or when organizational separation is itself an assurance mechanism.
Prefer one well-instrumented agent when reasoning is tightly coupled, all roles use the same evidence and tools, the coordinator must repeatedly reconstruct global state, or the task is short enough that delegation adds no meaningful capability.
The default should be the smallest number of decision-making components that makes responsibility clear.

# Part V: Applications and the current frontier

Representative systems as of August 2026

# Coding agents operate in an executable environment

A coding agent can observe repository state, search files, edit code, execute tests, inspect failures, and revise. This makes software engineering a natural agent domain:

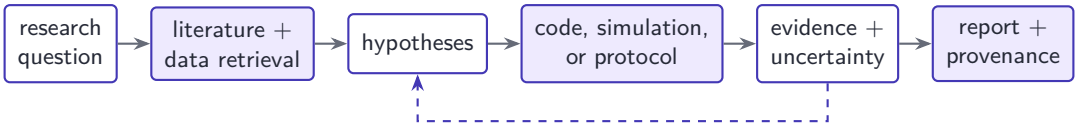
$$\text{issue} \to \text{localize} \to \text{edit} \to \text{test} \to \text{diagnose} \to \text{revise}.$$

- The environment supplies precise artifacts: files, diffs, logs, and test outcomes.
- Many actions are reversible through version control.
- Unit tests provide informative but incomplete feedback.
- Repository-scale tasks require state management beyond one context window.

Success on a test suite must be distinguished from correctness with respect to an incomplete specification. Evaluation should include cost and reproducibility, not only resolved-task rate.

Motivated by Kapoor et al., *AI Agents That Matter*; MLE-bench and software-agent benchmarks.

# Research agents connect information to executable work

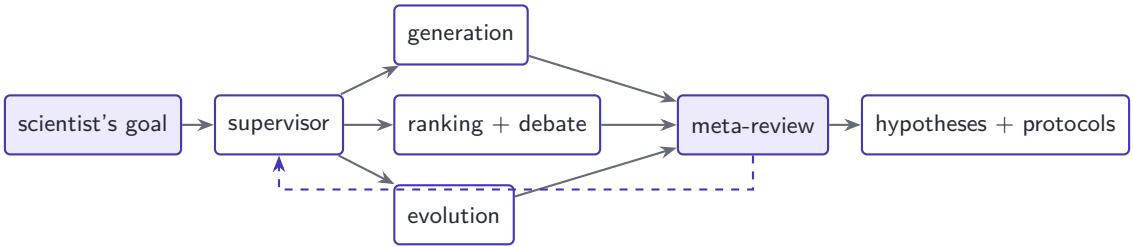



The system is agentic where intermediate evidence changes the next retrieval, computation, or experiment. It is scientifically useful only if claims remain connected to sources, code, measurements, and explicit uncertainty. Automating manuscript generation is easier than automating reliable discovery; the latter depends on evaluators and experimental interfaces.

Motivated by Agents for Science curriculum; EAIRA.

# Google AI co-scientist: iterative hypothesis search

The AI co-scientist is a multi-agent system for generating and refining scientific hypotheses and proposals. A supervisor allocates work among specialized generation, reflection, ranking, evolution, proximity, and meta-review agents.




The system uses test-time compute to generate, compare, and refine alternatives. Automated Elo-style ranking supports internal search, but human expert assessment and empirical validation remain essential.

Motivated by Gottweis and Natarajan, Google Research (2025).

# Virtual Lab: multi-agent design with physical validation

The Virtual Lab comprises an LLM principal-investigator agent, specialist scientist agents, and a human researcher. The team developed a computational nanobody-design pipeline using ESM, AlphaFold-Multimer, and Rosetta.

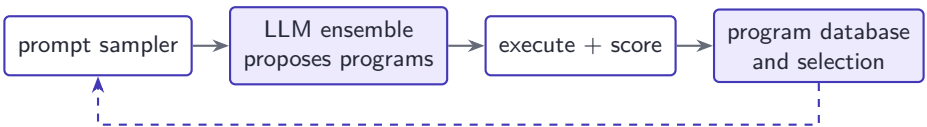
| Stage | Agentic contribution | External evidence |
|---|---|---|
| Team formation | select specialist roles and tools | human feedback on expertise |
| Design | propose and discuss nanobody candidates | protein-model and structural scores |
| Selection | compare candidates and pipeline outputs | computational ranking |
| Validation | transfer designs to laboratory work | binding experiments |

The system designed 92 nanobodies; laboratory experiments identified functional binders, including candidates with improved binding profiles for recent variants. The decisive evidence was physical measurement, not multi-agent agreement.

Motivated by Swanson et al., *Nature* 646, 716–723 (2025).

# AlphaEvolve: program search with automated evaluators

AlphaEvolve combines an ensemble of language models with evolutionary selection and objective program evaluators.




Reported applications include algorithm discovery, data-center and hardware optimization, matrix-multiplication algorithms, and compute-kernel improvement.
The architecture is particularly suitable where candidates are executable programs and progress can be scored with objective, repeatable evaluators. The evaluator converts open-ended generation into measurable search.

Motivated by Google DeepMind, *AlphaEvolve* (2025).

# AI Scientist-v2: an end-to-end computational loop

AI Scientist-v2 uses agentic tree search and an experiment-manager agent to formulate hypotheses, design and execute experiments, analyze results, generate figures, and draft manuscripts.

**Architectural advances**

- reduced dependence on human-authored code templates;
- parallel search over research trajectories;
- executable experiments and artifact generation;
- vision-language feedback for figures and manuscripts.

**Interpretive limits**

- demonstrated primarily in machine-learning research;
- peer-review acceptance is a noisy proxy for validity;
- novelty, reproducibility, and scientific significance remain difficult to evaluate;
- automated research can scale flawed methodology as well as useful work.

Motivated by Yamada et al., *The AI Scientist-v2* (2025).

# Part VI: Evaluation, safety, and reproducibility

Assess the trajectory, not only the final response

# The evaluation unit is a trajectory

An agent run is

$$\tau = (o_0, m_0, a_0, o_1, m_1, a_1, \ldots, o_T, m_T, F_T),$$

where $F_T$ denotes final artifacts. Evaluation should inspect both outcome and process.

| Dimension | Question | Possible measure |
|---|---|---|
| Task success | was the external objective achieved? | verified success rate |
| Validity | are claims and artifacts supported? | expert or formal assessment |
| Efficiency | what resources were consumed? | tokens, calls, time, money, energy |
| Robustness | does performance persist? | seeds, perturbations, environment variants |
| Safety | were constraints respected? | policy violations and near misses |

Motivated by Kapoor et al. (2024); EAIRA.

# Benchmark accuracy is insufficient

Agent benchmarks often emphasize final success while omitting cost, reproducibility, infrastructure complexity, and sensitivity to task selection.
Kapoor et al. identify several methodological concerns:

- optimizing accuracy without jointly accounting for cost;
- conflating evaluation needs of model developers and downstream users;
- inadequate or absent holdout sets, enabling benchmark overfitting;
- inconsistent evaluation practices that impede reproduction;
- attributing gains to agent architecture without controlled ablations.

Evaluation should compare against simpler workflows using the same model and tools. Complexity is justified only by a reliable improvement at the relevant operating point.

Motivated by Kapoor et al., *AI Agents That Matter* (2024).

# Safety follows data and action boundaries

| Threat | Failure mechanism | System control |
|---|---|---|
| Prompt injection | retrieved data is treated as instruction | instruction-data separation, content labeling |
| Excessive authority | model can invoke unnecessary actions | least privilege, scoped credentials |
| Unsafe arguments | valid schema encodes harmful operation | semantic validation, approval gates |
| Data leakage | context or tools expose protected information | access control, redaction, audit |
| Unbounded execution | loops consume resources or repeat actions | budgets, timeouts, idempotency |

No single prompt can provide these guarantees. Security-relevant properties must be enforced at the tool, runtime, storage, and orchestration layers.

# Observability is required for diagnosis

A useful execution record contains

$$\ell_t = (\text{run id}, t, \text{model version}, \text{prompt hash}, m_t, a_t, o_{t+1}, \text{cost}, \text{latency}, \text{policy decision}).$$

Logs should make it possible to reconstruct

- which evidence was available when an action was selected;
- which model, prompt, tool, and configuration produced each artifact;
- why an action was authorized or rejected;
- where state first diverged from a successful trajectory;
- whether a retry repeats the same side effect.

Observability is not equivalent to storing every hidden intermediate token. The objective is an auditable system trace with relevant state, actions, evidence, and decisions.

# Reproducibility requires versioned artifacts

Agentic experiments should version at least

- model and inference configuration;
- prompts, tool schemas, and orchestration code;
- retrieved sources and database snapshots;
- code, environments, dependencies, seeds, and hardware;
- intermediate and final artifacts;
- human interventions and approval decisions.

Because trajectories are stochastic, one successful run is not a stable capability estimate. Report distributions across runs and characterize common failure modes.
For science, the final manuscript is not sufficient provenance. The executable and evidential chain must remain available for independent inspection.

Motivated by EAIRA; AI Agents That Matter; Agents for Science evaluation curriculum.

# A design checklist

Before introducing agentic control, specify

1. the external objective and acceptance evidence;
2. observations, state representation, and provenance model;
3. admissible action classes and least-privilege permissions;
4. tool schemas, execution boundaries, and side-effect controls;
5. planning, memory, verification, and termination policies;
6. human decision boundaries and escalation rules;
7. cost, latency, horizon, retry, and parallelism budgets;
8. trajectory-level evaluation against simpler baselines;
9. logging, reproducibility, rollback, and incident response.

If these elements cannot be stated clearly, the system is not ready for extended autonomy.

# Closing perspective

An LLM provides a flexible conditional policy over text and structured actions. Agentic AI begins when that policy is embedded in an explicit interaction process with state, tools, feedback, and termination.

The key engineering problem is **control allocation**: which choices benefit from model-directed adaptation, which guarantees must be implemented deterministically, which evidence verifies progress, and where human authority remains necessary.

Compound and multi-agent systems can increase capability through retrieval, computation, specialization, and search. They also introduce new interfaces through which errors, costs, and security failures propagate.

Use the simplest architecture that satisfies the task. Additional model calls, tools, memory, and agents should be justified by measurable gains in success or assurance.

# Selected sources

| Theme | Representative source |
|---|---|
| Curriculum | Agents for Science, https://agents4science.github.io/Class/curriculum.html |
| Transformer and LLMs | Vaswani et al. (2017); Brown et al. (2020); Ouyang et al. (2022) |
| Compound AI systems | Berkeley AI Research (2024); Lewis et al. (2020) |
| Reasoning and acting | Yao et al., ReAct (2022/2023) |
| Agent architectures | Cheng et al. (2024); AutoGen; Model Context Protocol |
| Scientific agents | Google AI co-scientist; Virtual Lab; AlphaEvolve; AI Scientist-v2 |
| Multi-agent failures | Cemri et al. (2025) |
| Evaluation | Kapoor et al., AI Agents That Matter; EAIRA |

Primary links used throughout: https://arxiv.org/abs/1706.03762, https://arxiv.org/abs/2210.03629, https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, https://arxiv.org/abs/2407.01502, https://arxiv.org/abs/2503.13657.